本文从NF4量化的原理开始，记录了自己一步步将CUDA编写的NF4反量化kernel性能优化至超越bitsandbytes库的水准。包括测试程序、使用ncu分析性能瓶颈等。

全部源码：GitHub，知乎博客：知乎

什么是NF4量化?

传统的 INT4 量化由于只有 16 个可表示的离散值，在线性映射下无法有效捕捉权重分布的细微差别。为了突破这一限制，4 位正态浮点量化（NF4）应运而生。它是针对深度学习模型权重特有的统计属性——零中心正态分布而设计的非线性数据类型。

通常 4 位量化（如 INT4）是**线性**的，就像一把普通的尺子，刻度均匀（比如：0, 1, 2, 3...）。但研究发现，大模型的权重分布并不均匀，它们大多集中在 0 附近，呈**正态分布（高斯分布）**。传统的线性量化对所有区域赋予相同的分辨率，这在零附近的“稠密区”会导致严重的信息丢失，而在尾部的“稀疏区”则浪费了表示能力。

NF4 的核心思想是利用分位数量化来构建**非线性映射**，确保每个量化“桶”（Bin）在正态分布下具有相等的概率质量。通过这种方式，NF4 能够将更多的表示位数分配给 0 附近的权重，而对边缘处的极端权重使用较宽的分辨率。

NF4 table

那么 NF4 是怎么来划分这把刻度不均匀的尺子的呢?

NF4 Table 的生成遵循以下步骤：

1. 等概率切分：想象正态分布的曲线下方是一个“蛋糕”。NF4 会根据数学上的“信息论最优”原则，把这个蛋糕切成 16 块，并确保**每一块的面积（即概率质量）完全相等**。
2. 寻找代表值：在每一块“蛋糕”里找一个最能代表这一块的数值。数学上是通过标准正态分布的**分位数函数（Q 函数）**来计算的
3. 归一化与对齐：将算出的 16 个代表值缩放到 $[-1, 1]$ 之间，并进行微调，确保其中有一个值是精确的 0，方便模型处理稀疏数据。

那么它的作用是什么呢?

它的核心作用是**“用极少的位数，保留最核心的信息”**

- 因为 4 位只能代表 16 个数字。NF4 这把尺子在 **0 附近的刻度非常密集**（因为大部分权重在这里），而在远离 0 的地方刻度很稀疏。这比刻度均匀的 INT4 精度高得多。

- 在显存里，我们不再存复杂的浮点数，只存 **0 到 15 这 16 个索引**。每个索引只占 4 位 。
- 当模型计算时，它拿着这个 4 位的索引去 NF4 Table 里查表，瞬间就能找回它对应的那个高精度浮点数 。

双量化

NF4 的另一大重点就是**双量化**机制。

第一层量化

在对一个权重矩阵进行 NF4 量化时，执行路径如下：

1. **分块处理**：将张量展平后，划分为大小为 64 的连续块。
2. **绝对最大值搜索**：在每个块内找到权重的绝对最大值 $absmax_q$(一级量化缩放因子，每块一个）。
3. **局部缩放**：将块内的所有权重除以$absmax_q$，使其全部落在 $[-1, 1]$ 区间内。
4. **查找表映射**：对于标准化后的每一个权重，在 NF4 查找表中寻找与其最接近的值，并存储该值对应的 4 位索引（0-15）。在该项目中，我们将两个4-bit索引打包成一个字节写入内存中。

第二层量化

块量化虽然提高了精度，但也带来了显著的额外内存开销。如果每个长度为 64 的块都存储一个 32 位的浮点缩放因子（FP32），那么平均每个参数将增加 $32/64 = 0.5$ 位的额外存储成本 。对于一个 65B 模型，这 0.5 位的额外负担相当于数 GB 的显存占用。

双量化通过对这些缩放因子进行二次量化来解决这一问题

1. **二次分组**：将第一层产生的所有 FP32 缩放因子收集起来，以每 256 个缩放因子为一组，找出最大值$absmax2$作为缩放因子
2. **局部缩放**：将块内的所有权重除以$absmax2$，使其全部落在 $[-1, 1]$ 区间内。
3. **8-bit 浮点量化**：将这些处理后的缩放因子量化为 8 位浮点数（FP8），通常选用对 8 位量化几乎无损的格式 。

反量化

在模型运行推理时，GPU 无法直接对 4 位索引或 8 位量化后的常数进行加法或乘法运算。因此，所有的计算实际上都是在 FP16 或 BF16 精度下进行的。这意味着权重必须在从显存读取到计算核心的过程中，通过硬件寄存器或共享内存进行“即时反量化”

当神经网络需要执行一个线性层的计算 $Y = W \cdot X$ 时，反量化流程如下：

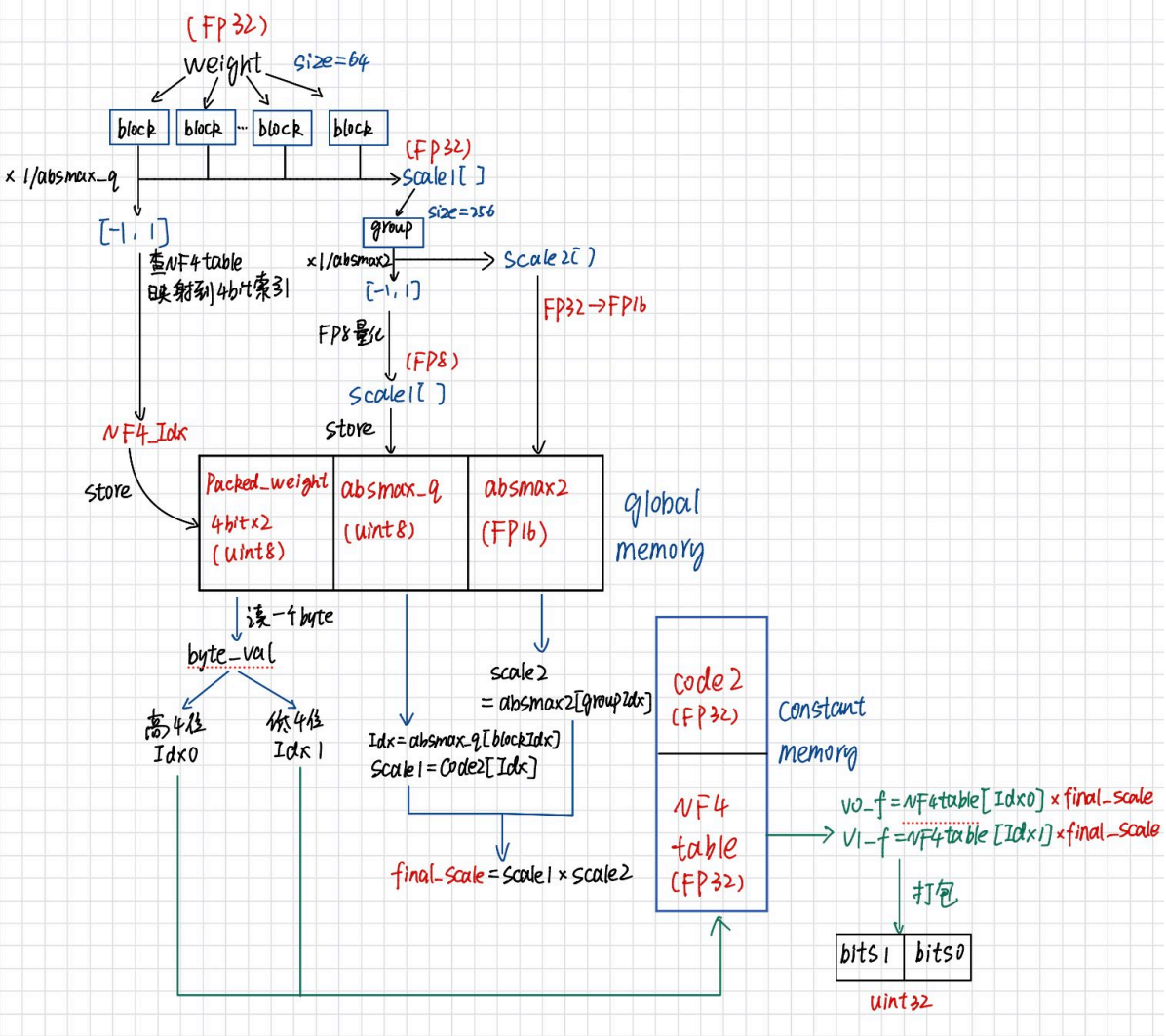
1. **获取二阶常数**：首先读取存储在显存中的$absmax2$（FP16），得到二级缩放因子$scale2 = absmax2[group_idx]$
2. **解压块缩放因子**：读取 8 位的量化缩放因子$absmax_q$，利用二阶常数将其还原为原始块缩放因子 $scale1$：

   $$scale1 = code2[absmax_q[block_idx]] \times absmax2[group_idx] + offset$$

   并计算出最终的缩放因子：$scale_{final} = scale1 \times scale2$
3. **查找 NF4 表值**：从显存中读取一个字节，也就是两个NF4索引(idx)。在NF4 table中根据这两个索引查找对印的值，利用最终的缩放因子$scale_{final}$将其还原成BF16：

   $$NF4_table[idx] \times scale_{final}$$

总流程：



CUDA Kernel代码逐步优化

naive

一个线程处理1-byte。现在测试用的矩阵大小都是4096×4096

这里我最初的想法是，像nf4 tabel 这样只有16个元素的数组，如果放进global memory中频繁地读取，那内存访问的开销就太大了。于是，我就写成了内联函数，这样在kernel中每个线程就会直接从寄存器中读取，岂不是快到飞起？这里犯了一个很大的错误，我们后面再讲。

```
__constant__ float c_code2[256];

// NF4 table
__device__ __forceinline__ float get_nf4_value(uint8_t idx)
{
    switch(idx) {
        case 0:  return -1.00000000f;
```

```
        case 1:  return -0.69619280f;
        case 2:  return -0.52507305f;
        case 3:  return -0.39491749f;
        case 4:  return -0.28444138f;
        case 5:  return -0.18477343f;
        case 6:  return -0.09105004f;
        case 7:  return  0.00000000f;
        case 8:  return  0.07958030f;
        case 9:  return  0.16093020f;
        case 10: return  0.24611230f;
        case 11: return  0.33791524f;
        case 12: return  0.44070983f;
        case 13: return  0.56261700f;
        case 14: return  0.72295684f;
        case 15: return  1.00000000f;
        default: return  0.0f;
    }
}

__global__ void dequantize_nf4_kernel
(
    const uint8_t*  __restrict__ packed_weights,
    const uint8_t*  __restrict__ absmax_q,
    const half*     __restrict__ absmax2,
    uint32_t*       __restrict__ output_packed,
    int num_bytes,
    int block_size,
    int group_size,
    float offset
)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= num_bytes) return;

    int element_idx = tid * 2;
    int block_idx = element_idx / block_size;
    int group_idx = block_idx / group_size;

    float scale_1 = c_code2[absmax_q[block_idx]] + offset;
    float scale_2 = __half2float(absmax2[group_idx]);
    float final_scale = scale_1 * scale_2;

    uint8_t byte_val = packed_weights[tid];
    uint8_t idx_0 = byte_val >> 4;
    uint8_t idx_1 = byte_val & 0x0F;
```

```
    float v0_fp32 = get_nf4_value(idx_0) * final_scale;
    float v1_fp32 = get_nf4_value(idx_1) * final_scale;

    __nv_bfloat16 v0_bf16 = __float2bfloat16(v0_fp32);
    __nv_bfloat16 v1_bf16 = __float2bfloat16(v1_fp32);

    uint16_t bits_0 = *reinterpret_cast<unsigned short*>
(&v0_bf16);
    uint16_t bits_1 = *reinterpret_cast<unsigned short*>
(&v1_bf16);

    output_packed[tid] = ((uint32_t)bits_1 << 16) |
(uint32_t)bits_0;
}
```

```
void nf4_dequantize_cuda
(
    std::vector<uint8_t>& h_packed_weights,
    std::vector<uint8_t>& h_absmax_q,
    std::vector<uint16_t>& h_absmax2,
    std::vector<uint16_t>& h_code2,
    int64_t rows, int64_t cols, int32_t blocksize, int32_t
goupsize,float offset
)
{
    size_t num_bytes = h_packed_weights.size();
    size_t out_size = num_bytes * sizeof(uint32_t);

    float h_code2_f32[256];
    for(int i = 0; i < 256; ++i)
    {
        __half h_val = *reinterpret_cast<__half*>(&h_code2[i]);
        h_code2_f32[i] = (float)h_val;
    }
    CHECK_CUDA(cudaMemcpyToSymbol(c_code2, h_code2_f32,
sizeof(h_code2_f32)));

    uint8_t *d_packed, *d_absmax_q;
    half *d_absmax2;
    uint32_t *d_output;

    CHECK_CUDA(cudaMalloc(&d_packed, h_packed_weights.size()));
    CHECK_CUDA(cudaMalloc(&d_absmax_q, h_absmax_q.size()));
    CHECK_CUDA(cudaMalloc(&d_absmax2, h_absmax2.size() * 2));
```

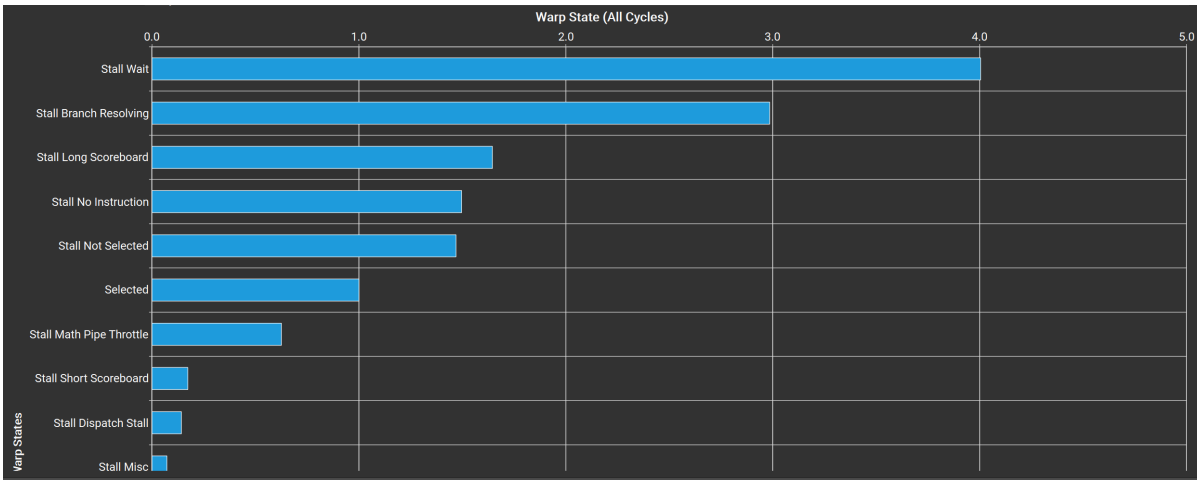
```
    CHECK_CUDA(cudaMalloc(&d_output, out_size));

    CHECK_CUDA(cudaMemcpy(d_packed, h_packed_weights.data(),
h_packed_weights.size(), cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_absmax_q, h_absmax_q.data(),
h_absmax_q.size(), cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_absmax2, h_absmax2.data(),
h_absmax2.size() * 2, cudaMemcpyHostToDevice));

    int threadsPerBlock = 256;
    int num_elements_vec = num_bytes;
    int blocksPerGrid = (num_elements_vec + threadsPerBlock - 1) /
threadsPerBlock;

    // warm up
    for (int i = 0; i < 5; ++i)
    {
        dequantize_nf4_kernel<<<blocksPerGrid, threadsPerBlock>>>
        (d_packed, d_absmax_q, d_absmax2, d_output, num_bytes,
blocksize, goupsize, offset);
    }

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start); // 开始记录
    std::cout << "Launch Kernel...\n";
    for (int i = 0; i < 10; ++i)
    {
        dequantize_nf4_kernel<<<blocksPerGrid, threadsPerBlock>>>
        (d_packed, d_absmax_q, d_absmax2, d_output, num_bytes,
blocksize, goupsize, offset);
    }
    cudaEventRecord(stop);  // 结束记录

    CHECK_CUDA(cudaEventSynchronize(stop)); // 等待 Event 完成

    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    // 计算带宽
    // 读取: Packed(1) + Indices(1) + Scales(2) + Code2(忽略不计)
    size_t total_read = h_packed_weights.size() +
h_absmax_q.size() + h_absmax2.size() * 2;
    // 写入: Output(4) (因为每个packed byte生成一个uint32)
```

```
    size_t total_write = num_bytes * 4;

    double total_bytes = (double)(total_read + total_write);
    double gb_per_sec = (total_bytes * 10 / 1e9) / (milliseconds /
1000.0);

    std::cout << "Kernel 耗时: " << milliseconds << " ms" <<
std::endl;
    std::cout << "有效带宽: " << gb_per_sec << " GB/s" <<
std::endl;

    // 保存时间与带宽
    std::ofstream timefile("./data/log/log_cpp.txt");
    timefile << milliseconds << "," << gb_per_sec;
    timefile.close();

    std::vector<uint32_t> h_output(num_bytes);
    CHECK_CUDA(cudaMemcpy(h_output.data(), d_output, out_size,
cudaMemcpyDeviceToHost));

    const std::string output_path = "./data/cpp_output.bin";
    std::ofstream outfile(output_path, std::ios::binary);
    outfile.write(reinterpret_cast<char*>(h_output.data()),
out_size);
    outfile.close();

    cudaFree(d_packed); cudaFree(d_absmax_q); cudaFree(d_absmax2);
cudaFree(d_output);
    cudaEventDestroy(start); cudaEventDestroy(stop);
}
```

运行结果:

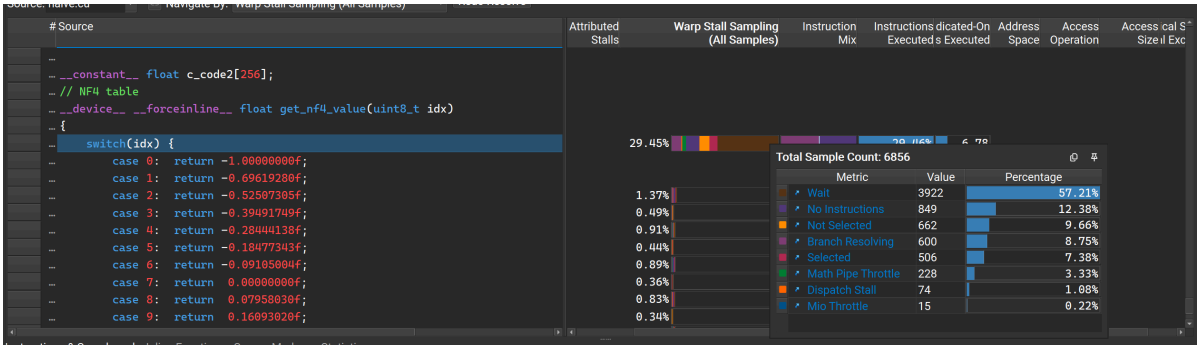
```
bnb耗时:2.08790ms, 带宽:202.15060GB/s
nf4 kernel耗时:6.23104 ms, 带宽:67.7371 GB/s
```

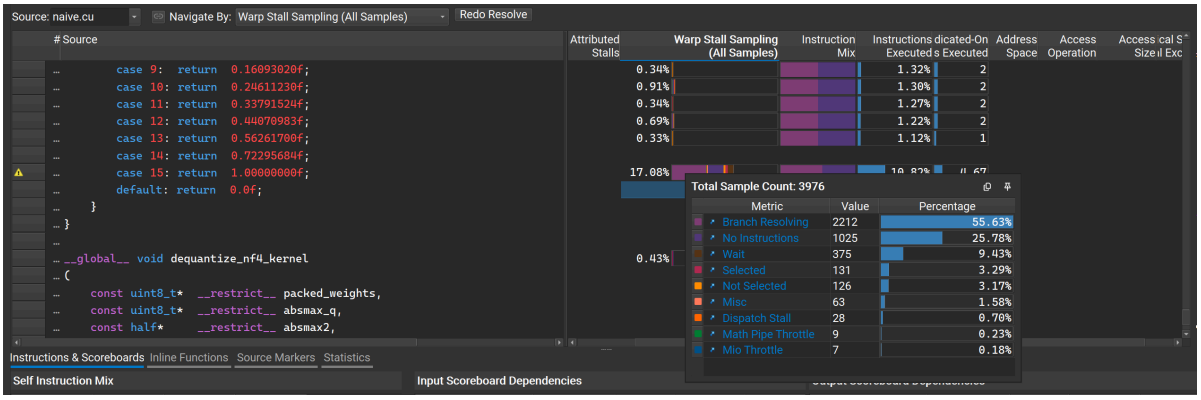
比bnb慢了这么多！我们打开ncu找找原因。



warp 全都在 stall wait 和 stall branch resolving，也就是大部分时间都在空闲！

我们打开source，看看问题具体出在哪一行代码上。





在switch-case的开头，warp stall 的主要原因是wait；而在结尾，warp stall 的主要原因则是branch resolving。

现在问题已经很清楚了，罪魁祸首就是这个switch-case导致线程束分化！每个线程负责的packed_weight对应的nf4索引很可能不相同，所以在访问nf4 table时就会发生线程束分化，导致一个warp中所有的线程都串行执行，所以才会出现warp stall！

这个地方坑了我好久，导致在用后面其他的优化方法时一直都收效甚微，但就是没想过问题出在这个自以为巧妙的内联函数上。

那就把nf4 table放在常量内存中吧！

```
__constant__ float c_code2[256];
// NF4 table
__constant__ float c_nf4[16] =
{
    -1.00000000f, -0.69619280f, -0.52507305f, -0.39491749f,
    -0.28444138f, -0.18477343f, -0.09105004f,  0.00000000f,
    0.07958030f,  0.16093020f,  0.24611230f,  0.33791524f,
    0.44070983f,  0.56261700f,  0.72295684f,  1.00000000f
};


__global__ void dequantize_nf4_kernel
(
    const uint8_t*  __restrict__ packed_weights,
    const uint8_t*  __restrict__ absmax_q,
    const half*     __restrict__ absmax2,
    uint32_t*       __restrict__ output_packed,
    int num_bytes,
    int block_size,
    int group_size,
    float offset
)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= num_bytes) return;

    int element_idx = tid * 2;
    int block_idx = element_idx / block_size;
    int group_idx = block_idx / group_size;

    float scale_1 = c_code2[absmax_q[block_idx]] + offset;
    float scale_2 = __half2float(absmax2[group_idx]);
    float final_scale = scale_1 * scale_2;

    uint8_t byte_val = packed_weights[tid];
    uint8_t idx_0 = byte_val >> 4;
    uint8_t idx_1 = byte_val & 0x0F;

    float v0_fp32 = c_nf4[idx_0] * final_scale;
    float v1_fp32 = c_nf4[idx_1] * final_scale;

    __nv_bfloat16 v0_bf16 = __float2bfloat16(v0_fp32);
    __nv_bfloat16 v1_bf16 = __float2bfloat16(v1_fp32);
```

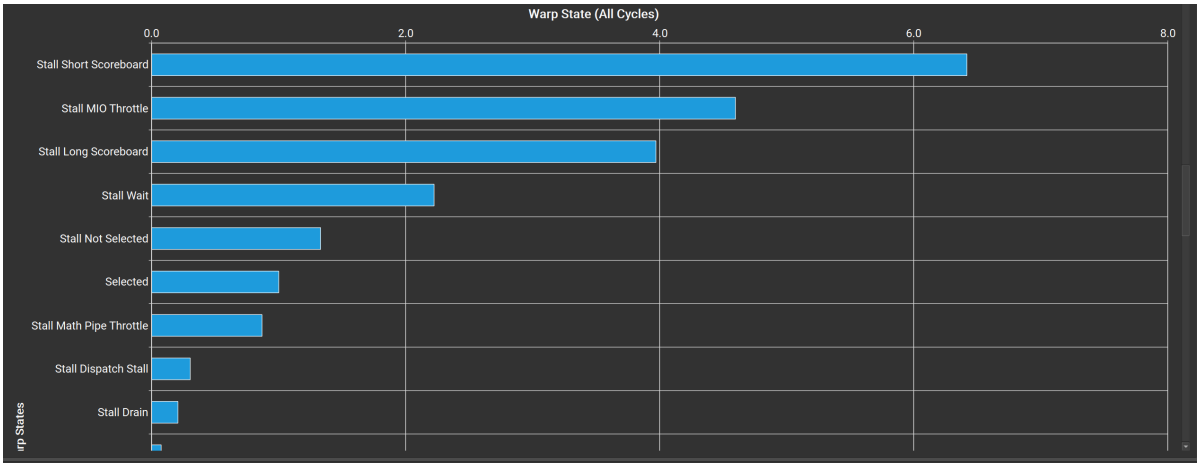
```
    uint16_t bits_0 = *reinterpret_cast<unsigned short*>
(&v0_bf16);
    uint16_t bits_1 = *reinterpret_cast<unsigned short*>
(&v1_bf16);

    output_packed[tid] = ((uint32_t)bits_1 << 16) |
(uint32_t)bits_0;
}
```

运行结果：

```
Kernel 耗时: 4.68173 ms
有效带宽: 90.1531 GB/s
```

可以看出，相比于前者已经有了很大提升。

现在我们在ncu中看看。



stall wait已经大大降低，branch resolving更是完全消除了。但是，新的问题出现了：stall short scoreboard、stall long scoreboard和stall MIO throttle很高！

什么是 stall short scoreboard？它像是在告诉你："你的指令虽然发射出去了，但是结果还没算完，后面的指令急着要用这个结果，所以大家都得等。"当 GPU 遇到数据依赖（Data Dependency）时，就会发生 Stall。也就是考研408计组指令流水线那一章节中的"数据冒险"，当前指令需要用到上一条指令运算的数据，所以导致指令流水线的阻塞。

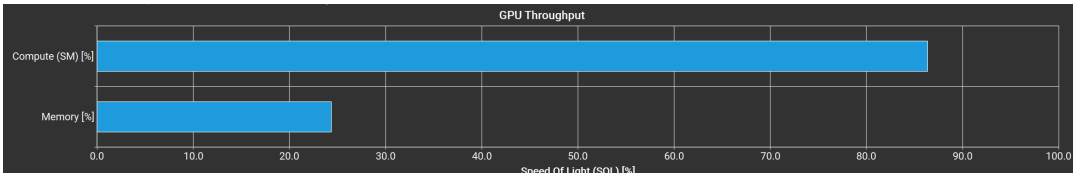
那相对的，stall long scoreboard是什么呢？**Long Scoreboard** 负责追踪那些**延迟极高、且延迟不可预测**的操作。在 GPU 架构中，这几乎总是指向两个地方：

1. **Global Memory (显存)**：去 VRAM 里捞数据（LDG/STG）。
2. **Local Memory (局部内存)**：也就是寄存器溢出（Register Spill）后去的那个地方，本质上也是显存。

Stall Long Scoreboard的意思就是：你的 Warp 想要执行某条指令，但这条指令需要的数据还在从显存传输到寄存器的路上。由于路途遥远（需要 400~800 个时钟周期），Warp 只能挂起（Stall），在那干瞪眼。

当你看到 **Stall Long Scoreboard** 高时，先看 **Throughput**。

- 带宽满了？—— 恭喜你，压榨干了硬件性能。
- 带宽没满？—— 赶紧改代码，上向量化，上 ILP，上预取，别让 GPU 闲着



我们一看带宽，低得离谱。这也为我们下一步优化指明了方向。

最后来解释 stall MIO throttle。

当前端指令发射速度太快了，后端的内存队列已经被塞满时，就会出现Stall MIO Throttle (Memory I/O Throttle)。

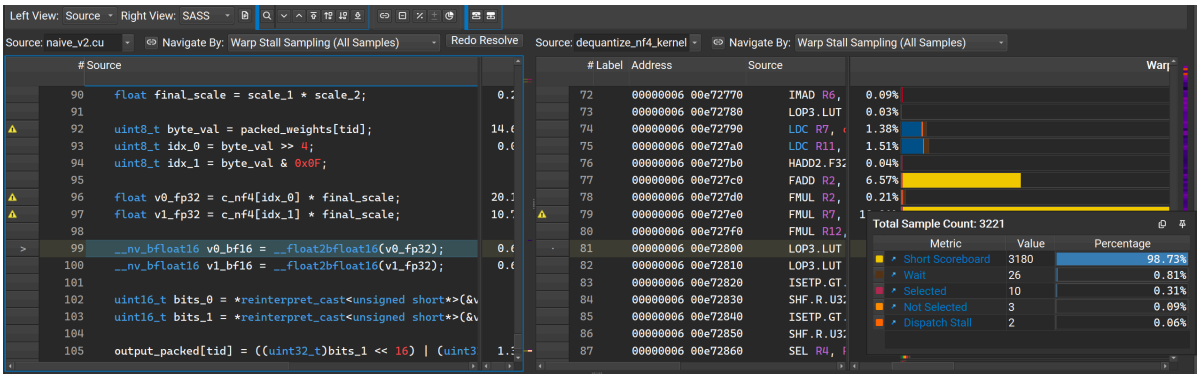
在 NVIDIA GPU 架构（Volta, Turing, Ampere+）中，MIO是连接 SM与内存系统的关键管道。它负责处理一下几类主要的请求：

- **Store (写入)**：往 Global Memory、Local Memory 写数据。
- **Constant (常量)**：读取 Constant Memory。
- **Shared Memory (部分)**：虽然 Shared 有专门的 LDS 管道，但在某些架构或特定操作（如 Atomics）下也会经过 MIO。

当你的 SM 疯狂地发射 `STG` (Store Global) 或 `LDC` (Load Constant) 指令，而后端还没处理完之前的请求时，MIO 的输入队列就会爆满。此时，MIO 会向前端发送信号："停！别再发了！"

于是，Warp 被迫 **Stall MIO Throttle**。

现在，我们在代码中具体分析以上瓶颈出现在哪。



我们注意到这几行发生的 stall short scoreboard 极高。

```
__nv_bfloat16 v0_bf16 = __float2bfloat16(v0_fp32);
__nv_bfloat16 v1_bf16 = __float2bfloat16(v1_fp32);

uint16_t bits_0 = *reinterpret_cast<unsigned short*>(&v0_bf16);
uint16_t bits_1 = *reinterpret_cast<unsigned short*>(&v1_bf16);
```

为什么呢？关键就在于`__float2bfloat16`这条指令。这是一条需要特殊功能单元（SFU）或复合 ALU 操作的指令，它不是瞬间完成的，通常需要 4 到 10 个时钟周期的延迟。

而GPU在刚刚发射完这条指令后，紧接着就要使用这条指令运算的结果：

```
uint16_t bits_0 = *reinterpret_cast<unsigned short*>(&v0_bf16);
```

这就产生了我们之前提到的“数据冒险”，当前指令所需要的数据是上一条指令的运行结果，从而导致了流水线阻塞。

那怎么消除stall short scoreboard呢？很简单，向量化！

假设一次性只处理一个float：

```
// 伪代码：处理 1 个元素
float v0 = ...;
// [T=0] 发射转换指令。需要 6 个周期才能算完
half h0 = __float2half(v0);

// [T=1] 立即使用 h0
// 调度器检查：h0 好了吗？没有（还要等 5 个周期）
// 结果：Stall Short Scoreboard (5 cycles)
store(h0);
```

现在，我们将粒度扩大 4 倍（处理 `float4` 或 4 个 `float`）：

```
// 伪代码：处理 4 个元素
float v0, v1, v2, v3;

// --- 阶段 1：批量发射计算（Producer）---

// [T=0] 发射 v0 转换。h0 开始计算（剩余 6 周期）
half h0 = __float2half(v0);

// [T=1] 发射 v1 转换。GPU 不会停！因为 h1 不依赖 h0。
// 此时 h0 剩余 5 周期。
half h1 = __float2half(v1);

// [T=2] 发射 v2 转换。流水线继续跑。
// 此时 h0 剩余 4 周期。
half h2 = __float2half(v2);

// [T=3] 发射 v3 转换。
// 此时 h0 剩余 3 周期。
half h3 = __float2half(v3);

// 此时，如果你还有其他的操作（比如地址计算），继续插在这里...

// --- 阶段 2：批量使用结果（Consumer）---

// [T=X] 使用 h0。
// 如果中间插入了足够的指令，h0 的 6 个周期延迟早就过去了！
// 调度器检查：h0 好了吗？好了！
// 结果：无 Stall！
store(h0);
store(h1);
...
```

通过向量化，我们用 `v1, v2, v3` 的计算时间，完美掩盖了 `v0` 的等待时间。这就叫 **Latency Hiding（延迟掩盖）**。

现在，下一步的优化方向已经很明确了，那就是向量化访存！

向量化访存

一个线程处理4-bytes

```
__global__ void dequantize_nf4_kernel
(
    const uint8_t*  __restrict__ packed_weights,
    const uint8_t*  __restrict__ absmax_q,
```

```
    const half*      __restrict__ absmax2,
    uint32_t*        __restrict__ output_packed, // 注意：这里虽然是指
针，但我们会强转写 int4
    int num_bytes, // 这里的 num_bytes 指的是 input 的字节数
    int block_size,
    int group_size,
    float offset
)
{
    // 每个线程处理 4 个输入字节（即 8 个 NF4 权重）
    // 所以总线程数只需要是 num_bytes / 4
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid * 4 >= num_bytes) return;

    // 向量化读取：一次吞下 4 个字节（32-bit）
    uint32_t packed_4bytes = reinterpret_cast<const uint32_t*>
(packed_weights)[tid];

    // 准备 128-bit 的输出容器（4 个 uint32_t，每个 uint32_t 包含 2 个
bf16）
    uint32_t out_vec[4];

#pragma unroll
    for (int i = 0; i < 4; ++i) {
        uint8_t byte_val = (packed_4bytes >> (i * 8)) & 0xFF;

        // 计算当前权重的全局索引
        int element_idx = (tid * 4 + i) * 2;
        int block_idx = element_idx / block_size;
        int group_idx = block_idx / group_size;

        float scale_1 = c_code2[absmax_q[block_idx]];
        float scale_2 = __half2float(absmax2[group_idx]);
        float final_scale = scale_1 * scale_2;

        // 解码两个 NF4
        uint8_t idx_0 = byte_val >> 4;
        uint8_t idx_1 = byte_val & 0x0F;

        float v0 = c_nf4[idx_0] * final_scale;
        float v1 = c_nf4[idx_1] * final_scale;

        __nv_bfloat16 b0 = __float2bfloat16(v0);
        __nv_bfloat16 b1 = __float2bfloat16(v1);
```

```
        uint16_t bits_0 = *reinterpret_cast<unsigned short*>(&b0);
        uint16_t bits_1 = *reinterpret_cast<unsigned short*>(&b1);

        // 打包结果存入临时数组
        out_vec[i] = ((uint32_t)bits_1 << 16) | (uint32_t)bits_0;
    }

    // 向量化写入
    reinterpret_cast<int4*>(output_packed)[tid] =
*reinterpret_cast<int4*>(out_vec);
}
```

注意，由于这里每个线程处理了四个字节，所以blocksPerGrid的配置要做以下修改：

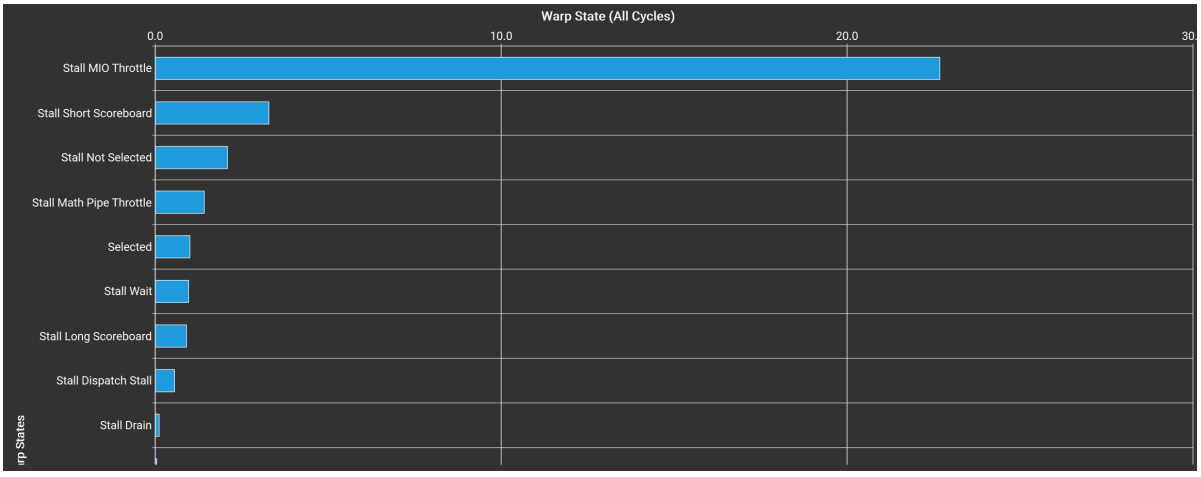
```
int threadsPerBlock = 256;
int num_elements_vec = (num_bytes + 3) / 4;
int blocksPerGrid = (num_elements_vec + threadsPerBlock - 1) /
threadsPerBlock;
```

运行结果：

```
Kernel 耗时: 4.46362 ms
有效带宽: 94.5584 GB/s
```

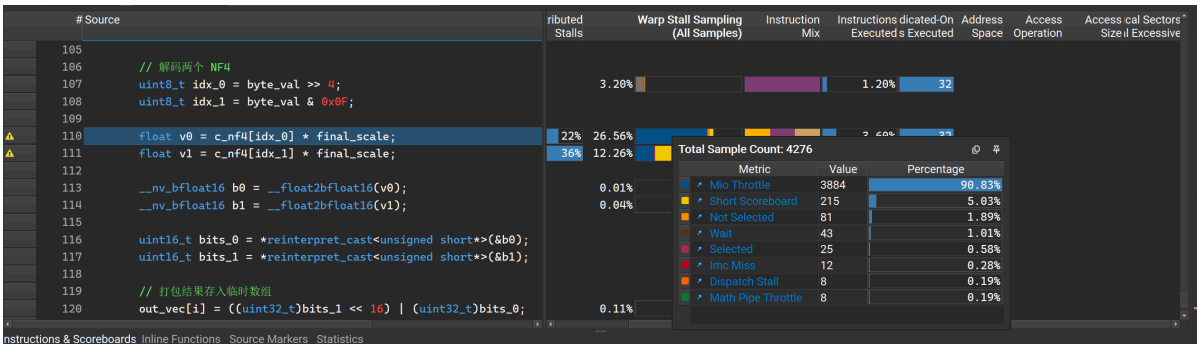
稍有提升，但是提升不大？

我们打开ncu看看。



可以看到，stall short scoreboard 已经大大下降了，这也印证了我们之前的观点：向量化访存可以消除数据冒险。

现在最大的问题就在于stall MIO throttle。



很明显，从处在常量内存中的`c_nf4`读取数据造成了很大的MIO Throttle。

常量内存缓存是为了“广播”设计的，不是为“随机查找”设计的。当一个 Warp 内的 32 个线程访问不同的常量地址时，硬件必须把这个操作拆解成多次串行事务，这会直接卡死 MIO 的读取管道。

shared memory

最好的解决办法就是使用 **shared memory**。

Shared Memory（LDS 单元）专为高并发随机访问设计，它有 32 个 Bank，能并行处理 32 个不同的地址请求。将小表从常量内存搬运到 Shared Memory，就把压力从 MIO 管道转移到了 LDS 管道。这是一种典型的“负载均衡”策略——不要让 MIO 一个人干所有的活。

```
__global__ void dequantize_nf4_kernel
(
    const uint8_t*  __restrict__ packed_weights,
    const uint8_t*  __restrict__ absmax_q,
    const half*     __restrict__ absmax2,
    uint32_t*       __restrict__ output_packed,
    int num_bytes,
    int block_size,
    int group_size,
    float offset
)
{
    __shared__ float s_nf4[16];

    int tx = threadIdx.x;
    if (tx < 16)
    {
        s_nf4[tx] = c_nf4[tx];
    }
    __syncthreads();

    // 每个线程处理 4 个输入字节 (即 8 个 NF4 权重)
```

```
    // 所以总线程数只需要是 num_bytes / 4
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid * 4 >= num_bytes) return;

    // 向量化读取：一次吞下 4 个字节 (32-bit)
    uint32_t packed_4bytes = reinterpret_cast<const uint32_t*>
(packed_weights)[tid];

    // 准备 128-bit 的输出容器 (4 个 uint32_t，每个 uint32_t 包含 2 个
bf16)
    uint32_t out_vec[4];

#pragma unroll
    for (int i = 0; i < 4; ++i)
    {
        uint8_t byte_val = (packed_4bytes >> (i * 8)) & 0xFF;

        // 计算当前权重的全局索引
        int element_idx = (tid * 4 + i) * 2;
        int block_idx = element_idx / block_size;
        int group_idx = block_idx / group_size;

        float scale_1 = c_code2[absmax_q[block_idx]];

        float scale_2 = __half2float(absmax2[group_idx]);
        float final_scale = scale_1 * scale_2;

        // 解码两个 NF4
        uint8_t idx_0 = byte_val >> 4;
        uint8_t idx_1 = byte_val & 0x0F;

        float v0 = s_nf4[idx_0] * final_scale;
        float v1 = s_nf4[idx_1] * final_scale;

        __nv_bfloat16 b0 = __float2bfloat16(v0);
        __nv_bfloat16 b1 = __float2bfloat16(v1);

        uint16_t bits_0 = *reinterpret_cast<unsigned short*>(&b0);
        uint16_t bits_1 = *reinterpret_cast<unsigned short*>(&b1);

        // 打包结果存入临时数组
        out_vec[i] = ((uint32_t)bits_1 << 16) | (uint32_t)bits_0;
    }
```

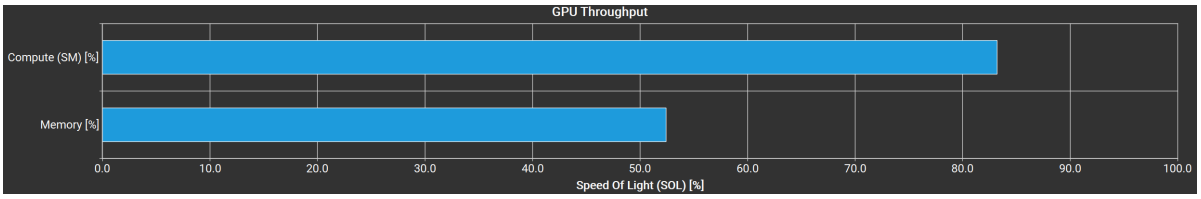
```
    // 向量化写入
    reinterpret_cast<int4*>(output_packed)[tid] =
*reinterpret_cast<int4*>(out_vec);
}
```

运行结果：

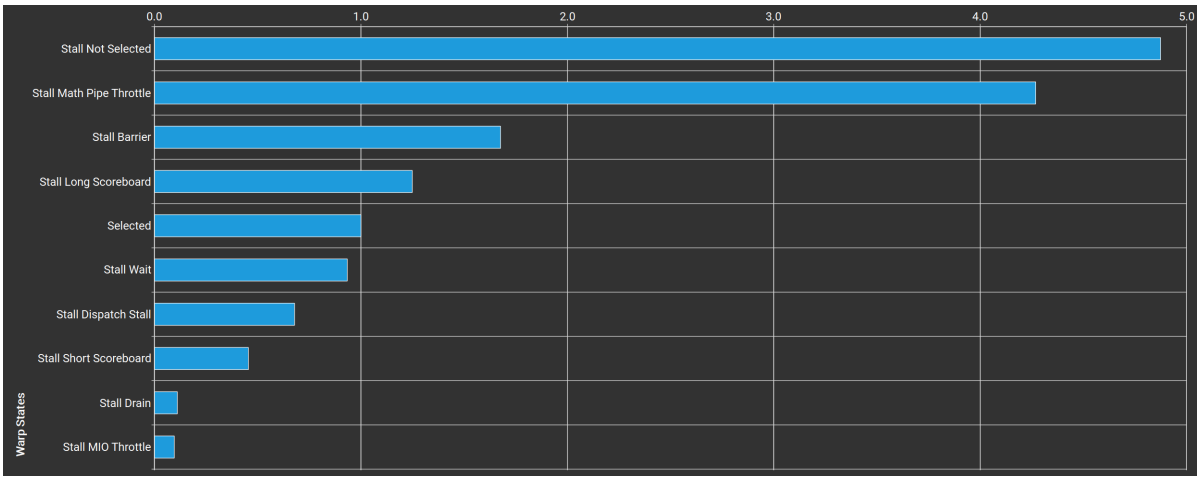
```
Kernel 耗时: 2.02854 ms
有效带宽: 208.067 GB/s
```

提升巨大！！！

我们来看ncu：



内存利用率提高了非常多。



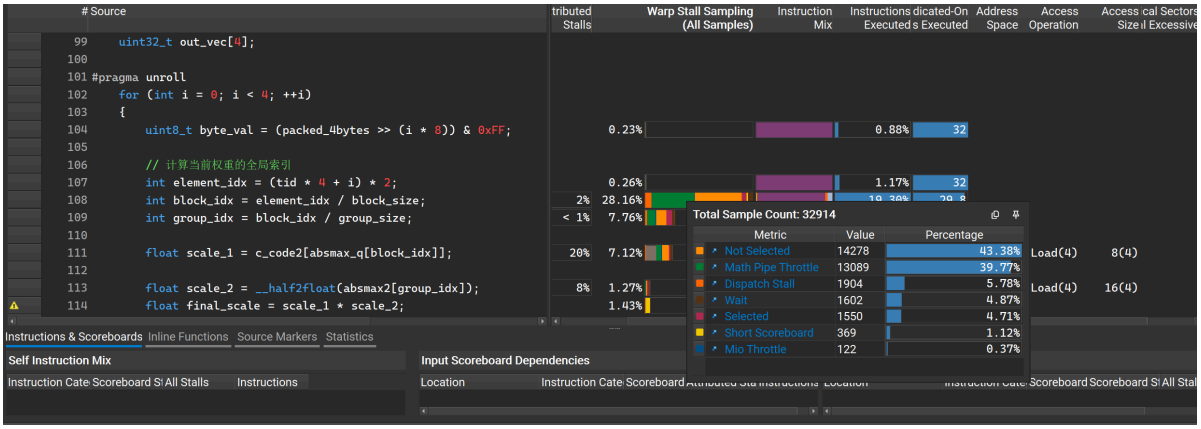
stall MIO Throttle已经降至几乎没有。

但是似乎出现了新的问题：stall not selected 和 stall math pip throttle 飙升！

stall math pip throttle，即数学管道阻塞。出现的原因一般是计算单元(ALU)计算数据的速度跟不上传输数据的速度。

而stall not selected呢？Warp Scheduler 说："手里有 10 个 Warp 都准备好干活了（没有内存依赖，数据都在寄存器里），但我每个周期只能安排 1 个 Warp 进流水线。剩下的 9 个，对不起，由于名额限制，你们只能未被选中。"这通常是 **Stall Math Pipe Throttle 的副作用**。因为数据来得太快（Shared Memory），所有的 Warp 都处于Ready状态。但是计算管道（Pipe）堵了（Math Throttle）。Scheduler 看着一堆 Ready 的 Warp，却因为硬件资源（ALU）繁忙或发射带宽限制，无法让它们全部运行。

我们在source中看看：



罪魁祸首就是108、109行这两条除法指令。

数除法是GPU 的天敌！ 如果 `block_size` 不是编译期常量且编译器无法证明它是 2 的幂，整数除法会被编译成几十条指令（软件模拟除法），极其消耗 ALU。但是在NF4量化中，`block_size`和`group_size`一般取值都是 2 的整数次幂，我们把除法指令换成移位指令不就可以了？

ALU_optim

只需要改动这两行：

```
int block_idx = element_idx >> block_shift;
int group_idx = block_idx >> group_shift;
```

然后在kernel launch函数中：

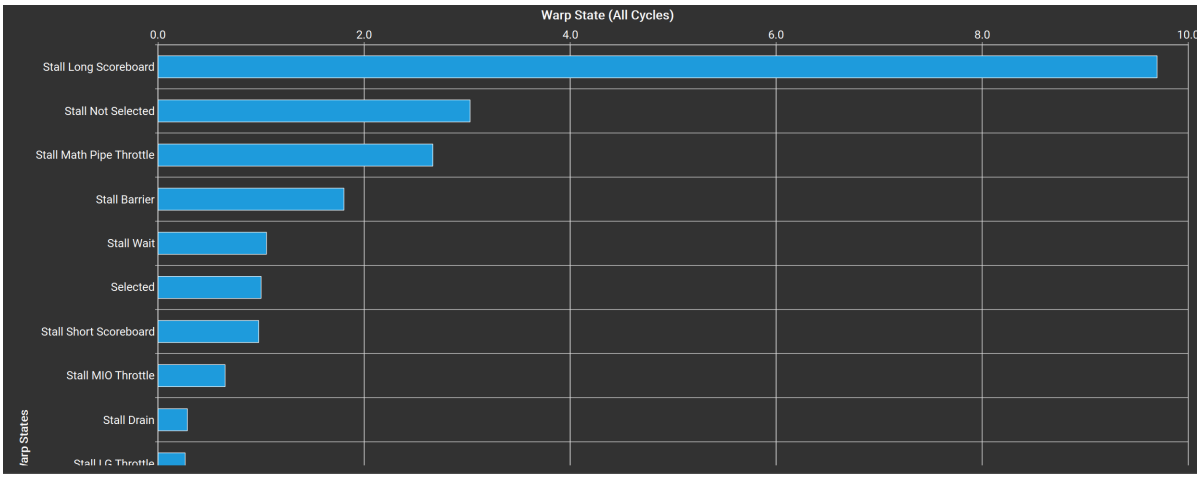
```
int block_shift = log2(blocksize);
int group_shift = log2(groupsize);
```

提前把移位的位数算好，传入kernel中。

运行结果：

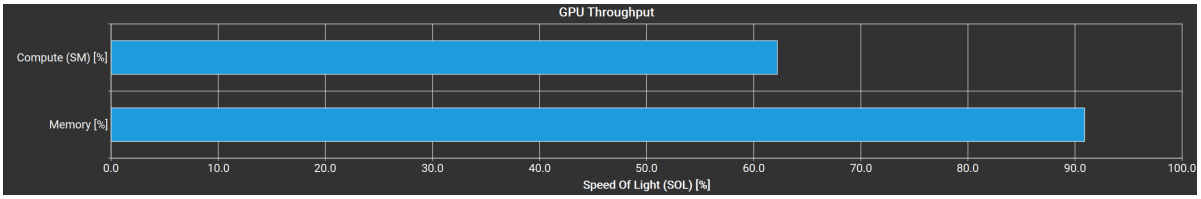
```
Kernel 耗时: 30.0431 ms
有效带宽: 224.782 GB/s
```

果然相比之前又有了提升！



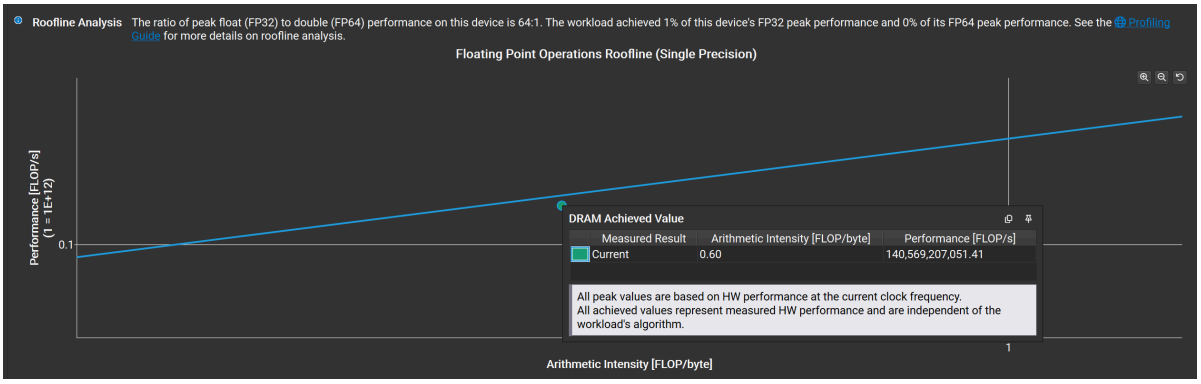
可以看到，stall not selected 和 stall math pip throttle都已经下降。

但是 stall long scoreboard 怎么这么高？别急，我们来看看 GPU Throughput：



内存带宽几乎拉满了！

再看看roofline：



已经几乎接近屋顶了！这说明我们的内存带宽已经到了硬件的极限，stall long scoreboard 高其实是优化成功的表现。

最后，我们再来看看在 A100 上使用16384×16384的大规模矩阵测试的表现

完整测试报告：

```
==================================================
误差分析报告
==================================================
平均绝对误差 (MAE):6.5369757067e-05
均方误差 (MSE):2.4582243441e-07
最大误差 (Max Diff):1.5625000000e-02
```

```
--------------------------------------------------
完全一致元素数:1043509311 / 1073741824
一致率:97.1844%
==================================================

 随机数据采样对比
-------------------------------------------------------------------
---------
Index      | Python (BF16)      | C++ (BF16)         | Diff

-------------------------------------------------------------------
---------
0          | 0.394531           | 0.394531           | 0.0000e+00

1          | 1.171875           | 1.171875           | 0.0000e+00

2          | -0.215820          | -0.215820          | 0.0000e+00

446162790  | -0.277344          | -0.277344          | 0.0000e+00

1066319814 | 0.000000           | 0.000000           | 0.0000e+00

230426597  | 0.205078           | 0.205078           | 0.0000e+00

951028163  | 0.322266           | 0.322266           | 0.0000e+00

857049811  | -0.566406          | -0.566406          | 0.0000e+00

527929503  | 0.718750           | 0.718750           | 0.0000e+00

23305680   | 0.000000           | 0.000000           | 0.0000e+00

168636035  | 0.134766           | 0.134766           | 0.0000e+00

660748068  | -0.625000          | -0.625000          | 0.0000e+00

801264382  | 0.098633           | 0.098633           | 0.0000e+00

-------------------------------------------------------------------
---------
bnb耗时:24.41320ms, 带宽:1106.47730GB/s
nf4 kernel耗时:21.62890ms, 带宽:1248.91000GB/s
```

赢。